

Sleep Apnea Detection via Respiratory & Oximetry Signals

A Comparative Study: CNN-Based Deep Learning vs. Classical Machine Learning

**Samuel Braga Marques - Miguel De Menezes Coelho - Juan Francisco Cortez Hervas - Popescu
Ianis-Stephan**

Deep Learning — Final Project Report
June 2026

1. Problem Statement

Obstructive sleep apnea (OSA) is a highly prevalent and clinically significant disorder characterized by repeated, temporary cessation of breathing during sleep. These episodes lead to oxygen desaturation, fragmented sleep architecture, and — over long periods — serious cardiovascular, metabolic, and neurocognitive consequences. Despite its prevalence, OSA remains substantially under-diagnosed because the gold standard for detection, Polysomnography (PSG), is expensive, inconvenient, and resource-intensive.

This project investigates the feasibility of automatic apnea detection using two non-invasive, lower-cost sensor modalities: (1) an inline pressure/flow mask sensor and (2) a neck-mounted pulse oximeter. Both are capable of capturing physiological signals related to respiratory effort and blood oxygen saturation, making them natural candidates for automated apnea screening.

The core research question is: can machine learning models — both deep neural networks trained end-to-end on raw time-series and classical feature-based classifiers — reliably distinguish between normal breathing and apnea events of varying severity from these sensor modalities? Specifically, we frame the task as a multi-class classification problem across three categories:

- Class 0 — Normal: regular, unobstructed breathing
- Class 1 — Apnea (≈ 10 s): short-duration apnea event
- Class 2 — Apnea (≈ 20 s): longer-duration apnea event

Beyond classification accuracy, a practical motivation for this work is evaluating whether simpler, interpretable classical models (ExtraTreesClassifier) can approach or match the performance of more complex deep architectures, potentially enabling deployment on resource-constrained devices.

2. Dataset Description

The dataset used throughout this project is the Respiratory Oximetry Apnoea dataset, publicly available on PhysioNet (physionet.org). It was collected in a controlled laboratory setting with 20 healthy adult subjects who underwent induced apnea events of known duration and airway resistance. Each subject performed a series of standardized tasks under three PEEP (positive end-expiratory pressure) levels: 0 cmH₂O, 4 cmH₂O, and 8 cmH₂O.

2.1 Sensor Modalities

Inline Mask Sensor (Inline_PQ_Data): A spirometry-grade pressure mask placed over the subject's face. It records two channels continuously: Gauge Pressure [cmH₂O] (absolute airway pressure) and Inspiratory Differential Pressure [cmH₂O] (pressure drop across a flow resistor). The native sampling rate is approximately 250 Hz. Normal breathing is characterized by rhythmic oscillations in both channels, while apnea events manifest as sustained low-amplitude, near-flat signal segments due to the absence of airflow.

Neck Pulse Oximeter (Neck_Pulse_Oximeter_Data): A custom neck-worn photoplethysmography (PPG) sensor containing 8 optical channels — 4 Red (PD1–PD4) and 4 Infrared (PD1_9–PD4_9). These channels measure light reflectance at the neck, capturing pulse waveforms and, indirectly, respiratory-modulated pulsatility changes. The device records at approximately 250 Hz. During apnea, characteristic changes in waveform amplitude, inter-beat interval, and SpO₂-related ratio can be observed.

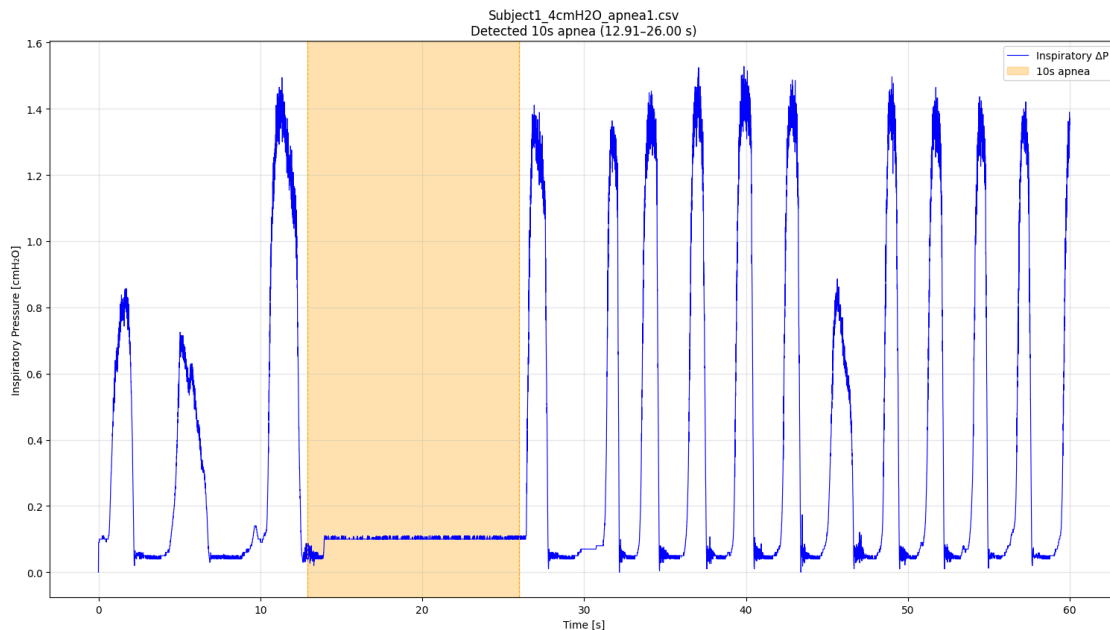


Figure 2. ~10s Apnea detected on the 4cmH₂O mask recording of the first subject.

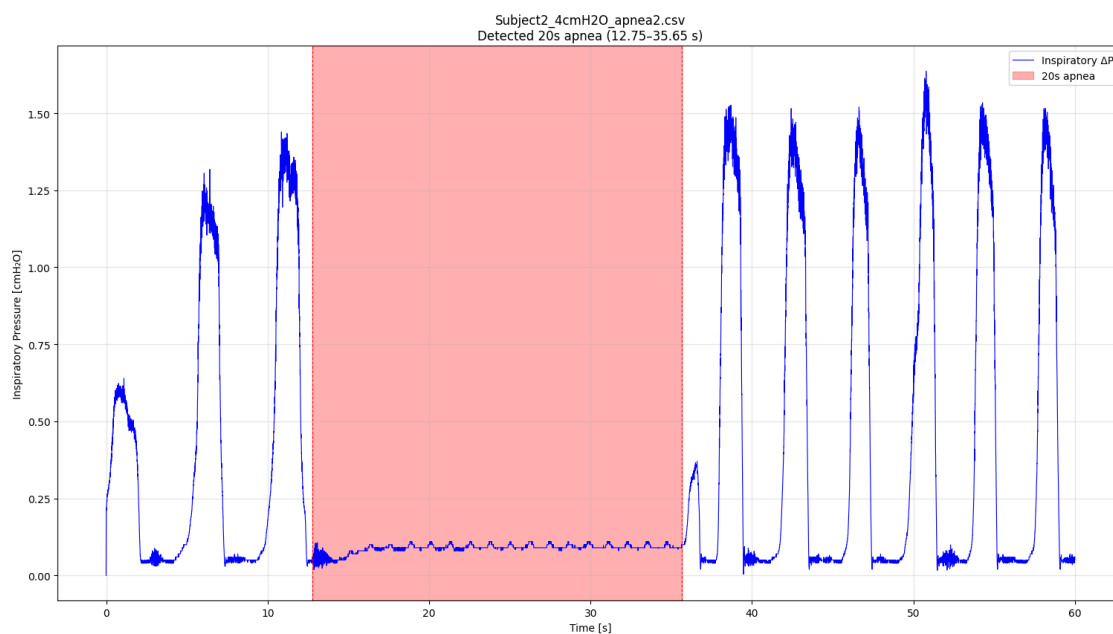


Figure 3. ~20s Apnea detected on the 4cmH2O mask recording of the second subject.

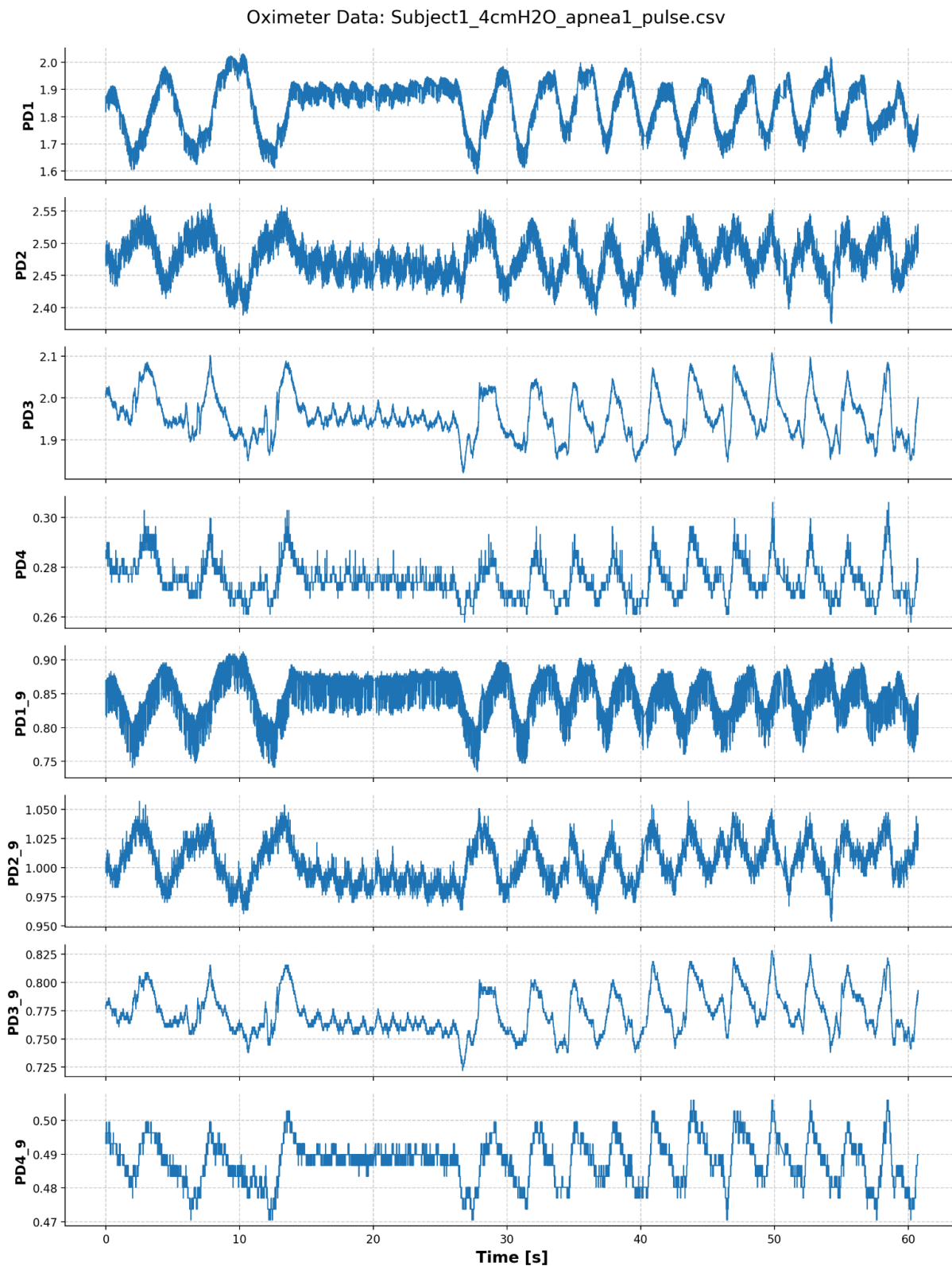


Figure 1. Neck oximeter data from the first subject during a ~10s apnea

2.2 Task Protocol

Each subject completed up to 7 tasks per PEEP level. The relevant task categories for this project are:

Task ID	PEEP Level	Condition	Label
0 / 1	0 cmH ₂ O	Normal breathing	Class 0 — Normal
1 / 2	4 cmH ₂ O	Normal breathing	Class 0 — Normal
2 / 3	4 cmH ₂ O	Apnea ~10 s	Class 1 — Apnea (10 s)
3 / 4	4 cmH ₂ O	Apnea ~20 s	Class 2 — Apnea (20 s)
4 / 5	8 cmH ₂ O	Normal breathing	Class 0 — Normal
5 / 6	8 cmH ₂ O	Apnea ~10 s	Class 1 — Apnea (10 s)
6 / 7	8 cmH ₂ O	Apnea ~20 s	Class 2 — Apnea (20 s)

Table 1. Task protocol and corresponding class labels used in classification.

2.3 Preprocessing & Windowing

Both model families apply a shared preprocessing philosophy, though the exact implementations differ:

- Resampling: all signals are resampled or padded to a uniform 250 Hz grid to ensure consistent temporal resolution across subjects and tasks.
- Low-pass filtering: a 4th-order Butterworth filter is applied zero-phase (via `filtfilt`) to remove high-frequency noise. The cutoff is 2.5 Hz for mask signals (capturing respiratory frequencies 0.1–2 Hz) and 5 Hz for the neck oximeter (broader to retain cardiac components).
- Apnea labeling: ground-truth apnea intervals are extracted by a signal-based detector (`apnea_detector.py`) that uses rolling standard deviation to locate low-activity segments whose duration matches the expected event duration from the filename. Intervals are stored in `mask_apnea_labels.csv`.
- Sliding window segmentation: Windows of defined sizes are extracted with subject-level train/val/test splits (subjects 1–14 / 15–17 / 18–20) to prevent data leakage.
- For the classical ExtraTrees baseline, the preprocessing differs from the CNN pipeline. Instead of producing many overlapping labeled windows, each 60-second trial is treated as one sample. The class label is taken from the filename metadata: `normal`, `apnea1`, or `apnea2`. The binary task maps `apnea1` and `apnea2` into a single `apnea` class, while the multiclass task keeps `normal`, `apnea1`, and `apnea2` as separate classes. The classical pipeline cleans invalid timestamps and missing values, then extracts fixed-window statistical and spectral features from the full trial.

2.4 Class Imbalance

Because each recording contains one apnea event of fixed duration within a 60-second session, normal-class windows significantly outnumber apnea windows. For the classical ExtraTrees model, class imbalance is handled at the trial level rather than the window level. Each full recording contributes one feature vector and one label. The model uses `class_weight='balanced'` so that minority classes are not ignored during tree construction. This is

especially important for macro F1 and balanced accuracy, which evaluate performance across classes equally.

3. Model 1: 1D Convolution Neural Network

Model 1 consists of a deep 1D CNN architecture (ApneaCNN1D, defined in `finalmodel.py`). It operates directly on raw, windowed time-series data without hand-crafted features, learning temporal patterns end-to-end through stacked 1D convolutions.

3.1 SWDataset — Sliding-Window Data Pipeline

The architecture utilizes the SWDataset class, which implements a PyTorch Dataset. For each subject and task, it loads the CSV, applies a low-pass Butterworth filter, and generates windows of SAMPLING_POINTS timesteps at a given stride (OFFSET). Each window is labeled as Normal (0), Apnea-10s (1), or Apnea-20s (2) based on its overlap with the detected ground-truth apnea interval. A window is assigned a positive apnea label only if it captures $\geq 80\%$ of the full apnea event duration, avoiding ambiguous partial-event windows. At retrieval time, each window is z-score normalized per-channel, making the model invariant to absolute sensor amplitude, and Gaussian noise is injected during training to discourage memorization

3.2 Advanced: ApneaCNN1D ([finalmodel.py](#))

The ApneaCNN1D model is built to maximize representational capacity while employing strong anti-overfitting measures based on training curve diagnostics. It utilizes a single classification head and heavily regularizes a shared convolutional trunk:

Block	Configuration	Notes
Conv Block 1	in_channels $\rightarrow$ 32 filters, kernel=15, padding=7 $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ Dropout1d(0.10) $\rightarrow$ MaxPool(2)	Largest kernel for coarse real-world receptive field
Conv Block 2	32 $\rightarrow$ 64 filters, kernel=9, padding=4 $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ Dropout1d(0.15) $\rightarrow$ MaxPool(2)	Increased feature depth for intermediate patterns.
Conv Block 3	64 $\rightarrow$ 128 filters, kernel=5, padding=2 $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ Dropout1d(0.20) $\rightarrow$ MaxPool(2)	Peak representational capacity
Conv Block 4	128 $\rightarrow$ 64 filters, kernel=3, padding=1 $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ Dropout1d(0.20) $\rightarrow$ MaxPool(2)	Bottleneck compression to cut parameters and regularize
Global Pool	AdaptiveAvgPool1d(16)	Aggregates evidence across whole window to a fixed-size vector
FC-Binary head	Flatten $\rightarrow$ 128 $\rightarrow$ ReLU $\rightarrow$ Dropout(0.4) $\rightarrow$ num_classes (2 or 3)	Single classification output (Binary or Multiclass)

Table 3. Apnea CNN1D multi-task architecture.

Batch normalization stabilizes gradients in the deeper stack, while per-block Dropout1d zeroes out entire feature channels—an ideal granularity for convolutional features that directly fights overfitting, which is a significant challenge in this project. Kernel sizes intentionally shrink with depth ($15 \rightarrow 9 \rightarrow 5 \rightarrow 3$) because each MaxPool(2) operation halves the temporal resolution; thus, maintaining a consistent real-world receptive field requires a smaller kernel at deeper, coarser stages. The AdaptiveAvgPool1d(16) layer turns variable-length feature maps into a fixed-size vector, matching the task objective since an apnea event can occur anywhere within the window. Training utilizes the Adam optimizer ($\text{lr}=1\text{e-}3$) with a raised weight decay ($5\text{e-}4$) for a stronger L2 penalty, alongside a ReduceLROnPlateau scheduler. Overconfidence and class imbalance are explicitly penalized using a cross-entropy loss augmented by dynamically computed class weights (via a WeightedRandomSampler) and label smoothing (0.05).

3.4 Training Configuration

The training configuration is adapted depending on the specific classification task. After extensive testing, it was determined that different temporal receptive fields are optimal for binary versus multiclass prediction:

- **Binary Task (Normal vs. Apnea):** Works best when passed the whole recording (60s). Because the binary task only requires detecting the *presence* of an anomaly, viewing the entire macroscopic context of the trial allows the model to easily contrast the flatline event against the surrounding rhythmic breathing.
- **Multiclass Task (Normal vs. 10s vs. 20s):** Works best with a 45-second windowing approach and a 2-second train overlap (stride=500). Differentiating a 10s apnea from a 20s apnea requires the model to precisely measure the duration of the event. The sliding window provides the model with multiple shifted views of the exact apnea boundaries, effectively augmenting the dataset while capturing the critical transition points (start and end of the airflow cessation) necessary to separate the classes. Furthermore, test evaluation uses strictly non-overlapping windows to provide an honest, independent-sample estimate of generalization.

Hyperparameter	CNN1D (Baseline)
Input channels	2 (mask) or 8 (neck)
Window length	11,250 samples (45 s @ 250 Hz) for Multiclass
Train stride	500 samples (2 s overlap) to reduce redundancy
Test stride	11,250 samples (no overlap)
Batch size	16
Epochs	40 (with early stopping patience of 8)
Optimizer	Adam ($\text{lr}=1\text{e-}3$, $\text{weight_decay}=5\text{e-}4$)
Loss	Weighted CrossEntropy with Label Smoothing (0.05)
Train subjects	1–14 (14 subjects)
Val subjects	15–17 (3 subjects)
Test subjects	18–20 (3 subjects)

Table 4. Training hyperparameters for both CNN architectures.

4. Model 2: ExtraTreesClassifier(Classical Baseline)

Model 2 (`resp_oximetry_classical_baseline.py`) takes a classical machine learning approach: hand-crafted statistical features are extracted from both sensor modalities, then fed into an `ExtraTreesClassifier` (an ensemble of extremely randomized decision trees). This pipeline serves as an interpretable, computationally lightweight reference against which the CNN models are benchmarked.

4.1 Feature Engineering

Rather than learning from raw waveforms, the `ExtraTreesClassifier` ingests a rich set of descriptive statistics computed across multiple temporal windows within each 60-second trial. Six time windows are defined:

- `w00_10`: seconds 0–10 (early phase)
- `w10_25`: seconds 10–25 (transition)
- `w25_40`: seconds 25–40 (mid-trial)
- `w40_60`: seconds 40–60 (late phase)
- `w00_30`: seconds 0–30 (first half)
- `full`: entire 60-second recording

For each channel within each window, the `robust_stats()` function extracts 12 statistical descriptors:

Feature	Description
mean, std, median	Central tendency and dispersion
p05, p95, iqr, range	Percentile-based spread (robust to outliers)
rms	Root mean square (signal energy)
mad	Median absolute deviation
diff_abs_mean, diff_std	Mean and std of first differences (rate of change / variability)
slope	Linear trend over the window (via polyfit)

Table 5. Per-channel statistical features extracted in each time window.

For the two wider windows (`w00_30` and `full`), spectral features are additionally computed via FFT:

- `fft_dom_freq`: dominant frequency (0.05–8 Hz) — respiratory rhythm correlate
- `fft_resp_frac`: fraction of spectral power in 0.05–0.7 Hz (respiratory band)
- `fft_heart_frac`: fraction of spectral power in 0.7–3 Hz (cardiac band)

Additional cross-channel and temporal contrast features are computed for the mask sensor (gauge/differential pressure correlation, quiet fraction) and the neck oximeter (Red/IR ratio statistics, inter-channel correlations). Finally, early-to-late contrast ratios (std and mean-absolute-value) capture how signal characteristics change between the first and second halves of the trial — a particularly informative feature for apnea detection.

In total, the feature extraction pipeline produces several hundred features per trial, which are then assembled into a feature matrix and cached to disk (`features.csv`) for efficiency.

Notably, PEEP level is deliberately excluded from the feature set. Although the airway pressure setting is available in the file metadata, including it would let the classifier key on experimental-condition information rather than physiological waveform patterns. Excluding it keeps the baseline focused on signal-based classification and avoids leakage of the protocol structure into the model.

4.2 Model Architecture & Training

The classifier is a scikit-learn Pipeline consisting of:

- SimpleImputer (strategy='median'): handles any NaN features introduced by empty or short time windows.
- ExtraTreesClassifier: an ensemble of 600 (default, configurable) extremely randomized trees, with max_features='sqrt', min_samples_leaf=1, class_weight='balanced', and a fixed random state for reproducibility.

ExtraTreesClassifier was chosen over standard Random Forest because it introduces additional randomness in the split selection step (both the feature and the threshold are randomized), which typically reduces variance and speeds up training on high-dimensional feature spaces.

Why ExtraTrees instead of other classical models?

ExtraTrees was selected because it offers a strong balance between performance, simplicity, and interpretability for tabular feature-based classification. Logistic Regression would be simpler but may underfit because physiological features can interact in nonlinear ways. SVMs can model nonlinear boundaries but require feature scaling and careful hyperparameter tuning. Gradient boosting methods such as XGBoost or LightGBM may improve performance but add more dependencies and increase the risk of overfitting on a dataset with only 20 subjects. ExtraTrees does not require scaling, trains quickly on CPU, handles nonlinear feature interactions, and provides feature importance values for interpretation.

4.3 Evaluation Protocol: Leave-One-Subject-Out CV

A critical design choice in the ExtraTreesClassifier pipeline is the use of Leave-One-Subject-Out Cross-Validation (LOSO-CV). In each of 20 folds, one subject is held out as the test set while the remaining 19 are used for training. This protocol closely simulates a realistic deployment scenario where the model must generalize to a completely unseen individual, and avoids the optimistic bias of random subject mixing. Per-fold metrics (accuracy, balanced accuracy, macro F1) and a pooled confusion matrix are recorded, alongside predicted probabilities for ROC-AUC computation. Both binary (Normal vs. Apnea) and multiclass (Normal / Apnea-10s / Apnea-20s) tasks are evaluated in the same run.

5. Results

5.1 Model 1 — CNN Training Curves

Figure 1 displays the training and validation loss (left) and accuracy (right) curves for the ApneaCNN1D model trained on the mask sensor data for the multiclass task. The loss curves show a steady, synchronized decline, indicating that the strong regularization techniques (Dropout1d and weight decay) successfully prevented the model from memorizing the training data. The accuracy curve demonstrates rapid learning in the first 4 epochs, climbing from roughly 70% to a stable plateau around 90-95% on the validation set.

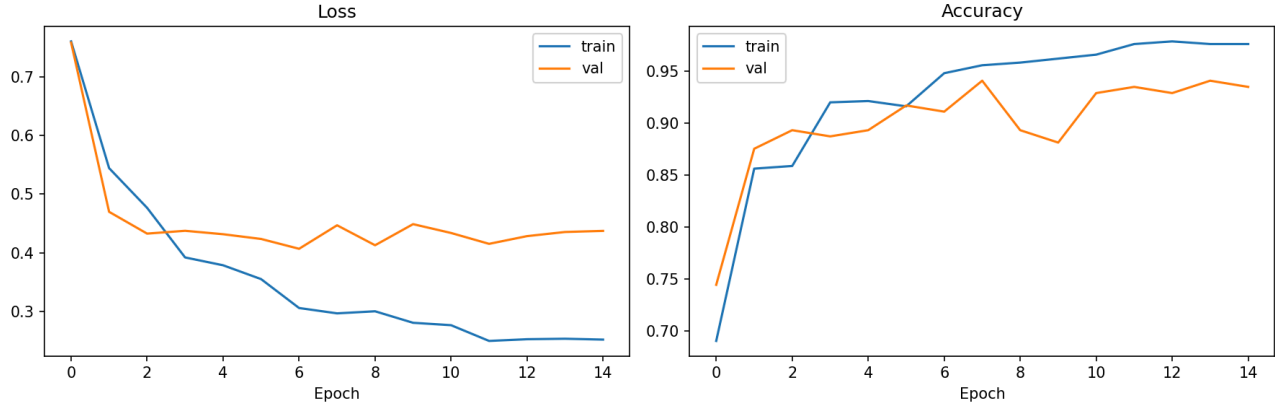


Figure 1. ApneaCNN1D training and validation loss (left) and accuracy (right) over 15 epochs, for the multi class task.

Figure 2 illustrates the training and validation loss (left) and accuracy (right) for the binary classification task. Because detecting the mere presence of an apnea is a significantly simpler problem, the model converges much faster. The validation accuracy reaches 100% almost immediately and remains stable, while the loss drops sharply and bottoms out near 0.15 within the first 5 epochs, triggering early stopping.

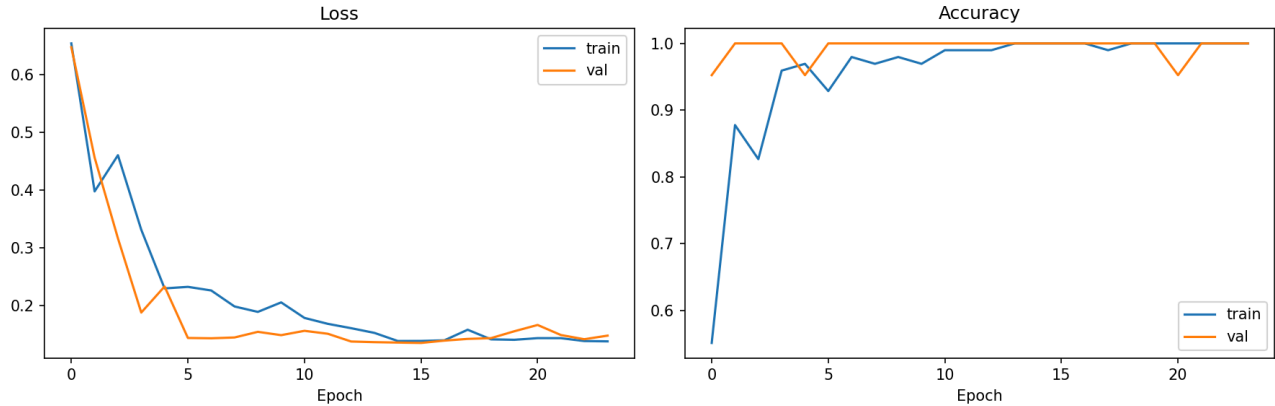


Figure 2. ApneaCNN1D training and validation loss (left) and accuracy (right) over 15 epochs, for the binary classification task.

5.2 Model 1 — Test Performance

Metric	Multi Class	Binary
Sensor	Mask (2 ch)	Mask (2 ch)
Val Accuracy (final epoch)	~90%	~100%
Test Accuracy (Unseen subjects)	~86%	~95%
Generalization on Neck Sensor	Poor / Failed to converge	Poor / Failed to converge
Best checkpoint	Saved on best val loss	Saved on best val loss

Table 6. Summary of CNN model performance characteristics.

Neck Oximeter Generalisation Failure: Despite high accuracy on the inline mask data, the CNN model ultimately failed to generalise and make correct predictions when trained on the 8-channel neck oximeter data across both tasks. A primary reason for this is the physiological delay in oxygen desaturation. While the inline mask directly and instantly measures the mechanical cessation of airflow via pressure drops, the neck PPG sensor relies on detecting secondary cardiovascular responses (such as SpO2 drops and pulse rate variations). During the short, artificially induced apneas in this protocol (10s to 20s), blood oxygen levels often do not drop significantly or rapidly enough to form a clear, generalisable pattern within the raw time-series window. Consequently, the CNN struggled to find a universal "apnea signature" in the high-frequency cardiac noise of the PPG channels, leading it to overfit on patient-specific waveform artefacts rather than learning the target pathology.

5.3 Model 2 — ExtraTreesClassifier LOSO-CV Performance

The ExtraTreesClassifier was evaluated using Leave-One-Subject-Out CV, which provides an unbiased estimate of generalisation to new subjects. Results for both binary and multiclass tasks are summarised below.

Metric	Binary Task	Multiclass Task
Accuracy	0.8857	0.7786
Balanced Accuracy	0.8729	0.7583
Macro F1	0.8802	0.7604
ROC-AUC	0.9632 (OvR)	0.9195 (OvR macro)
Evaluation protocol	LOSO-CV (20 folds)	LOSO-CV (20 folds)
# Estimators	600 ExtraTrees	600 ExtraTrees

Table 7. ExtraTreesClassifier LOSO-CV performance (measured across all 20 subject folds).

The binary task (Normal vs. Apnea) is considerably easier than the 3-class problem, as the model only needs to detect the presence of apnea rather than distinguish its severity. The multiclass task degrades gracefully, with the dominant confusion expected between the two apnea classes (10 s vs. 20 s), which can look similar in their early seconds.

Unlike the CNN, the ExtraTrees classifier is not trained over epochs and therefore does not produce epoch-based training and validation loss curves. Its generalization is evaluated using

Leave-One-Subject-Out cross-validation, fold-level metrics, confusion matrices, ROC-AUC, and feature importance analysis.

6. Comparison

Criterion	Model 1 (CNN)	Model 2 (ExtraTreesClassifier)
Input	Raw time-series windows (end-to-end)	Hand-crafted statistical + spectral features
Architecture	Stacked 1D Conv + FC heads	ExtraTrees ensemble (600 trees)
Evaluation	Fixed train/val/test subject split	Leave-One-Subject-Out CV (20 folds)
Multiclass accuracy	~85%	77.9%
Binary accuracy	~90%	88.6%
Training cost	High (GPU recommended, but fast due to low data amount)	Low (minutes on CPU)
Inference speed	Fast (GPU batch inference)	Fast (CPU, no GPU needed)
Interpretability	Low (black-box feature learning)	High (feature importances available)
Data requirements	Many windows needed (sliding stride)	Works with fewer trials (whole-trial features)
Generalization	Subject-split (potential leakage risk)	LOSO-CV (strict generalization)
Sensor coverage	Mask or Neck (separate models)	Both sensors fused in one pipeline

Table 8. Side-by-side comparison of Model 1 (CNN) and Model 2 (ExtraTreesClassifier).

6.1 Interpretation

While both model families achieved comparable multiclass accuracy and high binary accuracy, these top-line metrics mask critical differences in how each architecture handles distinct sensor modalities. The CNN demonstrated exceptional performance on the inline mask data—achieving near-perfect binary classification and 86% test accuracy on the multiclass task—by easily learning the morphological "flatline" signature of airflow cessation directly from raw waveforms.

However, the CNN's failure to generalise on the 8-channel neck oximeter data reveals a fundamental limitation of applying end-to-end deep representation learning to secondary physiological markers. Because the PPG sensor measures cardiovascular responses (e.g., pulse rate variability and SpO₂ changes) which are physiologically delayed and heavily corrupted by high-frequency cardiac noise, the CNN struggled to isolate a universal apnea signature. Instead, it overfitted to patient-specific waveform artefacts.

In contrast, the classical ExtraTreesClassifier pipeline has a more interpretable signal-fusion baseline across both modalities. By relying on explicitly engineered features—such as spectral power fractions and early-to-late contrast ratios—the classical model successfully bypassed the noise that confused the CNN. This highlights a crucial trade-off: while deep learning eliminates the need for manual feature engineering on direct, high-signal-to-noise modalities (like mask pressure), classical machine learning can be more practical for extracting delayed, indirect physiological events from raw waveforms, especially in data-constrained environments.

7. Discussion and Limitations

7.1 Discussion

This study demonstrates that both deep learning and classical machine learning approaches can successfully automate the detection of induced sleep apnea, provided the model architecture is appropriately matched to the underlying sensor modality. A key finding is that an end-to-end 1D CNN can achieve outstanding accuracy when processing direct mechanical airflow signals (inline mask), capitalising on its ability to learn local temporal patterns without human intervention.

Yet, this research also exposes the boundaries of "black-box" representation learning. CNN's inability to extract meaningful diagnostic criteria from the raw neck oximeter (PPG) waveforms underscores that not all physiological signals are amenable to raw time-series ingestion. When the physiological consequence of an event is delayed and buried within secondary systems—such as blood oxygen desaturation following an apnea—explicit feature engineering (e.g., computing Red/IR ratios and spectral densities for a ExtraTreesClassifier) provides a necessary inductive bias that a CNN cannot easily learn from a limited cohort of 20 subjects.

Ultimately, the dual-sensor approach confirms that automated apnea screening is highly feasible outside of traditional Polysomnography. However, for resource-constrained deployments, the computationally lightweight and highly interpretable ExtraTrees ensemble—which reliably handled both modalities—presents a more practical and robust solution than deep learning alone.

7.2 Limitations

- Controlled lab setting: all apnea events in this dataset are artificially induced under known conditions. Real-world sleep apnea is spontaneous, occurs in variable body positions, and is accompanied by other physiological fluctuations (arousal, movement artifacts, varying sleep stages) that are absent here. Generalizability to clinical OSA is therefore unknown.
- Small dataset: 20 subjects is a small sample for training deep learning models. The 14/3/3 split leaves only 3 test subjects, making the CNN test accuracy statistic potentially high-variance. The LOSO-CV protocol used for the ExtraTreesClassifier is more robust, but even 20 subjects may not represent the full population variability.
- Evaluation inconsistency: direct comparison of the two models is complicated by their different evaluation protocols (fixed split vs. LOSO-CV). A fair comparison would require evaluating both under LOSO-CV, which was not done here.
- Apnea label quality: the apnea interval labels are derived algorithmically from a signal-based detector (rolling standard deviation threshold), not from human-annotated gold-standard intervals. Any errors in the labeling cascade into both models' training and evaluation, potentially inflating or deflating performance estimates.
- Binary head underutilized: the AdvancedCNN1D's binary classification head is trained jointly but its performance is not separately reported or used for final inference. Investigating whether the binary head alone outperforms the multiclass head for simple presence/absence detection would be informative.
- No SpO2 computation: the neck oximeter data is used as raw PPG channels, but the ratio between Red and Infrared channels could in principle be converted to estimated SpO2 values, which are a primary clinical indicator of apnea severity. Incorporating SpO2 as an explicit feature could improve the ExtraTreesClassifier pipeline.

- Hyperparameter search: neither model underwent systematic hyperparameter optimization (e.g., learning rate search, number of trees, window size grid search). Performance could potentially be improved with structured tuning.

7.3 Future Work

Several directions could meaningfully extend this work:

- Transfer to real sleep recordings: evaluating models on open clinical PSG datasets (e.g., PhysioNet SHHS, MESA) with naturalistic apnea events would test true clinical utility.
- Attention mechanisms: adding temporal attention to the CNN (e.g., a squeeze-and-excitation block or self-attention layer) could help the model focus on the most apnea-discriminative segments within each 30-second window.
- Unified LOSO evaluation: re-evaluating the CNN under full LOSO-CV would enable a statistically fair comparison with the ExtraTreesClassifier.
- SpO2 integration: converting the Raw Red/IR PPG ratio to an explicit SpO2 estimate and using it as a feature could improve both models.
- Online detection: adapting the pipeline to streaming inference (causal filtering, no lookahead) would be a prerequisite for real-time wearable deployment.

8. References

- 1) Geurts, P., Ernst, D., & Wehenkel, L. (2006). Extremely randomized trees. *Machine Learning*, 63, 3–42. doi: 10.1007/s10994-006-6226-1.
- 2) Goldberger, A. L., Amaral, L. A. N., Glass, L., Hausdorff, J. M., Ivanov, P. C., Mark, R. G., Mietus, J. E., Moody, G. B., Peng, C. K., & Stanley, H. E. (2000). PhysioBank, PhysioToolkit, and PhysioNet: Components of a new research resource for complex physiologic signals. *Circulation*, 101(23), e215–e220. doi: 10.1161/01.CIR.101.23.e215.
- 3) Hanley, J. A., & McNeil, B. J. (1982). The meaning and use of the area under a receiver operating characteristic curve. *Radiology*, 143(1), 29–36. doi: 10.1148/radiology.143.1.7063747.
- 4) Hill, J., Guy, E. F. S., Clifton, J. A., Pretty, C., & Chase, J. G. (2026). Respiratory and Pulse Oximetry Waveforms from Healthy Adults During Simulated Apnoea Events (Version 1.0.0). PhysioNet. doi: 10.13026/s45r-k263.
- 5) Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. *Proceedings of the 32nd International Conference on Machine Learning*, 448–456.
- 6) Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *International Conference on Learning Representations*.
- 7) Kunjan, S., et al. (2021). The necessity of leave one subject out cross validation for EEG disease diagnosis. *International Conference on Neural Information Processing*. Springer.
- 8) Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., VanderPlas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- 9) Sokolova, M., & Lapalme, G. (2009). A systematic analysis of performance measures for classification tasks. *Information Processing & Management*, 45(4), 427–437. doi: 10.1016/j.ipm.2009.03.002.
- 10) Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15, 1929–1958.